

למידת חיזוק לתזמון משימות באשכולות מחשוב-על (HPC)

מסמך הסבר מקיף — רקע תיאורטי, שחזור המאמר, ומדריך קוד מלא

המאמר המשוחדר:

,Renato L. F. Cunha & Luiz Chaimowicz

"An SMDP approach for Reinforcement Learning in HPC cluster schedulers"

Future Generation Computer Systems 139 (2023) 239–252

DOI: 10.1016/j.future.2022.09.025

קוד המקור: github.com/renatolfc/sched-rl-gym (ענף smdp)

מנחה: פרופ' גל קמינקא, אוניברסיטת בר-אילן

תאריך: יוני 2026

1. הבעיה: תזמון משימות באשכול HPC
2. רקע: למידת חיזוק (Reinforcement Learning) מאפס
3. המאמר: מ-DeepRM ל-SMDP
4. יומן עבודה — מה נעשה כדי להריץ ולשחזר (יעד 1)
5. מדריך קוד מלא: sched-rl-gym קובץ-קובץ
6. סקריפט האימון `deepm-agent.py` — אלגוריתם REINFORCE לעומק
7. איך מריצים הכל — פקודות
8. תוצאות, מסקנות ביניים, וצעדים הבאים (יעדים 2–3)
9. נספח: מילון מונחים

1. הבעיה: תזמון משימות באשכול HPC

אשכול מחשוב-על (HPC cluster) הוא אוסף גדול של מעבדים שמשרת קהילת משתמשים. משתמשים שולחים **משימות (jobs)**: כל משימה מבקשת כמות מעבדים מסוימת ומשך ריצה מסוים. בדרך כלל אין מספיק משאבים פנויים להריץ הכל מיד, ולכן משימות ממתינות **בתור**. המתזמן (**scheduler**) הוא הרכיב שמחליט, בכל רגע, איזו משימה מהתור תקבל משאבים ותתחיל לרוץ.

החלטות התזמון משפיעות דרמטית על חוויית המשתמשים. המדד המקובל לאיכות תזמון הוא ה- **slowdown** של משימה — פי כמה המשימה "התעכבה" ביחס לאורך הריצה שלה:

$$\text{slowdown}(j) = (\text{finish}_j - \text{submit}_j) / \text{runtime}_j = (\text{wait}_j + \text{runtime}_j) / \text{runtime}_j$$

למשל: משימה של דקה שחיכתה 9 דקות קיבלה slowdown של 10 — חוויה איומה. משימה של 10 שעות שחיכתה 9 דקות בקושי הרגישה. לכן slowdown "מעניש" במיוחד עיכוב של משימות קצרות. קיים גם **bounded slowdown**, שמגביל את המכנה מלמטה (בדרך כלל ב-10 שניות) כדי שמשימות זעירות לא ינפחו את המדד (ממומש ב- `job.py` בשם `bounded_slowdown`).

1.1 מתזמנים קלאסיים (היוריסטיקות)

מתזמן	כלל ההחלטה	בקוד
FCFS / FIFO	לפי סדר הגעה	<code>fifo_scheduler.py</code>
SJF (Shortest Job First)	המשימה הקצרה ביותר קודם — מצוין ל- slowdown	<code>sjf_scheduler.py</code>
Backfilling / EASY	FCFS + "מילוי חורים": משימות קטנות עוקפות אם לא יעכבו את ראש התור	<code>backfilling_scheduler.py</code> , <code>easy_scheduler.py</code>
Packer	בוחר משימה שממקסמת ניצול משאבים (יישור עם הפנוי)	<code>packer_scheduler.py</code>
Random	בחירה אקראית — baseline תחתון	<code>random_scheduler.py</code>

הרעיון של המאמר: במקום לקבע כלל ידני, ניתן ל**למוד** את כלל ההחלטה מהנתונים עצמם בעזרת למידת חיזוק. המתזמן הנלמד יכול לתפוס דפוסים שכללים פשוטים מפספסים (למשל: "כדאי להשאיר חור פנוי כי בעוד רגע תגיע משימה גדולה").

2. רקע: למידת חיזוק מאפס

2.1 תהליך החלטה מרקובי (MDP)

למידת חיזוק (RL) עוסקת בסוכן שלומד להתנהג בסביבה מתוך ניסוי וטעייה. הפורמליזם המתמטי הוא **MDP — Markov Decision Process**, חמישייה $\langle S, A, P, R, \gamma \rangle$:

- **S — מרחב מצבים**: כל מה שהסוכן רואה. אצלנו: "צילום מצב" של האשכול והתור.
- **A — מרחב פעולות**: ההחלטות האפשריות. אצלנו: "תזמן את משימה מספר i מהתור" או "אל תעשה כלום".
- **$P(s'|s,a)$ — דינמיקת מעברים**: איך העולם מתקדם. אצלנו: הסימולטור — משימות מגיעות, רצות ומסתיימות.
- **$R(s,a)$ — תגמול**: משוב מספרי מיידי. אצלנו: שלילי, ביחס לכמות הממתינים (ראו 2.4).
- **γ — מקדם ניכיון (discount)**: כמה שווה תגמול עתידי לעומת מיידי ($0 \leq \gamma \leq 1$).

תכונת מרקוב: ההתפלגות של המצב הבא תלויה רק במצב הנוכחי ובפעולה — לא בהיסטוריה. **מדיניות** (**policy**) $\pi(a|s)$ היא הסתברות לבחור פעולה a במצב s. מטרת הלמידה: למצוא מדיניות שממקסמת את **התשואה** (**return**) — סכום התגמולים המנוכה:

$$G_t = r_t + \gamma \cdot r_{t+1} + \gamma^2 \cdot r_{t+2} + \dots = \sum_{k \geq 0} \gamma^k r_{t+k}$$

2.2 REINFORCE-i Policy Gradient

יש שתי משפחות עיקריות של אלגוריתמי RL: מבוססי-ערך (כמו Q-learning, שלומדים "כמה שווה" כל מצב) ומבוססי-מדיניות (שלומדים את π ישירות). הקוד שלנו משתמש בגישה השנייה: רשת נוירונים עם פרמטרים θ מקבלת מצב ופולטת התפלגות על פעולות — $\pi_\theta(a|s)$. משפט ה-Policy Gradient נותן את כיוון השיפור:

$$\nabla_\theta J(\theta) = E_\pi [\sum_t \nabla_\theta \log \pi_\theta(a_t|s_t) \cdot G_t]$$

אינטואיציה: **הגדל את ההסתברות של פעולות שהובילו לתשואה גבוהה, הקטן את אלה שהובילו לנמוכה**. אלגוריתם REINFORCE (Williams, 1992) מממש זאת ישירות: שחק אפיזודות שלמות עם המדיניות הנוכחית, חשב לכל צעד את G_t , ועדכן את θ בכיוון הגרדיאנט.

2.3 בעיית השונות וה-Baseline

הבעיה המעשית של REINFORCE היא **שונות גבוהה**: התשואות משתנות בפראות בין אפיזודות, והגרדיאנט רועש. הפתרון: להחסיר מ- G_t ערך ייחוס (b_t "baseline") שאינו תלוי בפעולה — זה לא מטה את הגרדיאנט אבל מקטין שונות מאוד:

$$\nabla_{\theta} J(\theta) = E [\sum_t \nabla_{\theta} \log \pi_{\theta}(a_t|s_t) \cdot (G_t - b_t)]$$

הביטוי ($G_t - b_t$) נקרא **advantage**: "כמה הפעולה הזו הייתה טובה יותר מהמוצע". בקוד שלנו (כמו ב-DeepRM המקורי), b_t הוא ממוצע התשואה בזמן t על פני כל האפיזודות באצווה — מימוש ב-`compute_baselines` שבסקריפט `deeprm-agent.py`.

2.4 ולמה SMDP? לב המאמר

ב-MDP רגיל הסוכן מחליט **בכל צעד זמן**. אבל בתזמון, ברוב צעדי הזמן אין בכלל מה להחליט — האשכול מלא ושום משימה לא נכנסת. הסוכן "מבזבז" אינטראקציות על מצבים שבהם לאף פעולה אין השפעה, והאשראי על תוצאות מתמרח על פני המון צעדים סרק — מה שמקשה על הלמידה.

Semi-MDP (SMDP) הוא הכללה שבה פעולה יכולה להימשך זמן משתנה: הסוכן מחליט רק ב"נקודות החלטה" אמיתיות, והזמן שביניהן נצבר עם ניכיון מתאים. אם בין שתי החלטות עברו τ צעדים עם תגמולי ביניים $r_1 \dots r_{\tau}$, התגמול האפקטיבי של המעבר הוא:

$$R = \sum_{k=1.. \tau} \gamma^{k-1} r_k, \quad \gamma \text{ והניכיון למצב הבא הוא}$$

זה בדיוק מה שהמאמר מציע לתזמון: **נקודת החלטה = רגע שבו משימה כלשהי בתור יכולה להיכנס לאשכול**. בין נקודות כאלה הסימולטור פשוט מקדם את הזמן בעצמו. בקוד זה ההבדל בין `TimeBasedDeepRmSimulator` (MDP) ל-`EventBasedDeepRmSimulator` (SMDP) — ראו פרק 5.6.

3. המאמר: מ-DeepRM ל-SMDP

3.1 הבסיס: DeepRM (HotNets 2016)

המאמר של Cunha & Chaimowicz בנוי על DeepRM של Mao et al — העבודה הראשונה שהראתה ש-RL עמוק יכול ללמוד תזמון. הרעיונות המרכזיים של DeepRM, שכולם ממומשים בריפוי:

- **המצב כתמונה:** מצב האשכול מקודד כמטריצה דמוית-תמונה. ציר אנכי = זמן עתידי (time_horizon, 20 צעדים), ציר אופקי = משאבים (10 מעבדים + 10 יחידות זיכרון). תא צבוע = תפוס. לצד "תמונת" האשכול מוצגות עד 5 משימות ראשונות מהתור (job slots) באותו קידוד, ועמודת backlog שסופרת כמה משימות נוספות ממתינות מעבר ל-slots.
- **פעולה:** מספר בין 0 ל-5: "הכנס את משימה i מה-slots לאשכול" (0-4) או "המתן" (5). אם הפעולה לא חוקית (המשימה לא נכנסת) — היא מנוטרלת והזמן מתקדם.
- **תגמול:** בכל צעד זמן, $R = -\sum_{j \in J} 1/T_j$ כאשר J היא קבוצת המשימות במערכת ו- T_j זמן הריצה של משימה j. הסכום הזה נבחר כי מקסום שלו שקול מתמטית למזעור ה-slowdown הממוצע (כל משימה "משלמת" $T/1$ בכל צעד המתנה — בדיוק התרומה שלה ל-slowdown). ממומש
`base.py : compute_reward -2`
- **אלגוריתם:** REINFORCE עם baseline לפי צעד-זמן (פרק 2.3).

3.2 התרומות של המאמר הנוכחי

1. **מידול SMDP:** הגדרת נקודות החלטה מבוססות-אירועים במקום צעדי זמן קבועים, עם צבירת תגמולים מנוכה (פרק 2.4). זו התרומה התיאורטית המרכזית, והיא משפרת מהירות למידה ואיכות.
2. **סביבת gym פתוחה ומודולרית** (sched-rl-gym עצמה) עם עומסים סינתטיים (מודל Lublin), עומסים אמיתיים (קבצי SWF מארכיון העומסים של האונ' העברית), ושני ייצוגי מצב (תמונה מלאה / קומפקטי).
3. **הערכה אמפירית** מול היוריסטיקות (SJF, packer, random, FCFS) ומול גרסת ה-MDP, גם על עומס סינתטי וגם על trace אמיתי (LANL-CM5).

4. יומן עבודה — מה נעשה כדי להריץ ולשחזר (יעד 1)

זה הפרק שמתעד בדיוק מה נעשה אצלך במחשב, באיזה סדר, ולמה. כל פעולה כאן ניתנת לשחזור עם הפקודות שבפרק 7.

4.1 זיהוי המאמר והקוד

הקישור של גל מוביל ל-ScienceDirect. דרך ה-DOI (S0167739X22003090) זוהה ה-DOI — `renato1fc/sched-rl-gym` — `j.future.2022.09.025/10.1016` והכותרת המלאה. המאמר מפנה לריפוי `smdp` (הענף שמכיל את מימוש ה-SMDP של המאמר; ענף `master` מכיל רק את גרסת ה-MDP).

4.2 סביבת העבודה

נמצאה סביבה וירטואלית (venv) קיימת עם Python 3.10 ועם `gym==0.21` — גרסת ה-gym הישנה שהקוד דורש (זו "ההתיישנות" שגל הזכיר: ה-API של gym השתנה מאז באופן שובר-תאימות, והספרייה הוחלפה ב-gymnasium). הותקנו בנוסף: PyTorch 2.12 (גרסת CPU), tensorboard (ללוגים של האימון) ו-matplotlib (לרינדור).

4.3 הקושי המרכזי: קומפילציה של מחולל העומסים על Windows

החבילה `parallelworkloads` היא קוד C/C++ עטוף ב-Cython שמממש את מודל Lublin99 — מודל סטטיסטי מפורסם שמייצר עומסי עבודה ריאליסטיים (התפלגויות של זמני הגעה, גדלים וזמני ריצה שכולו מול מערכות אמיתיות). הקוד המקורי נכתב ללינוקס ומשתמש בפונקציות POSIX שלא קיימות בקומפיילר של מיקרוסופט (MSVC):

- `drand48()` / `lrand48()` / `srand48()` — מחולל מספרים אקראיים
- `bzero()` — איפוס זיכרון

הבנייה נכשלה עם שגיאות `error C3861: identifier not found`. הפתרון: נכתב קובץ header אחד, `rand48_compat.h`, שמומש בו המחולל בדיוק לפי מפרט POSIX:

```

/* drand48 is a 48-bit Linear Congruential Generator:
 *   X(n+1) = (a*X(n) + c) mod 2^48,  a = 0x5DEECE66D, c = 0xB
 *   srand48(seed) sets X = (seed << 16) | 0x330E
 */
static uint64_t _rand48_state = 0x330E;

static void srand48(long seed) {
    _rand48_state = (((uint64_t)(uint32_t)seed) << 16) | 0x330E;
}

static double drand48(void) {
    _rand48_state = (0x5DEECE66DULL * _rand48_state + 0xBULL)
        & ((1ULL << 48) - 1);
    return (double)_rand48_state / (double)(1ULL << 48);
}

```

למה זה חשוב לשחזור: אילו היינו מחליפים את drand48 ב- rand() רגיל של C, הקוד היה מתקמפל — אבל סדרת המספרים האקראיים הייתה שונה, ולכן העומסים שנוצרים היו שונים מאלה של המאמר, וכל השוואה הייתה חסרת משמעות. המימוש לפי המפרט מבטיח שעם אותו seed נקבל בדיוק את אותן משימות כמו בלינוקס. זו נקודה מתודולוגית ששווה להציג לגל: שחזור מדעי דורש זהות ברמת הביטים של מקור האקראיות.

ה-header הוכלל בשני קבצים: m_lublin99.c (מודל Lublin) ו- est_model.cc (מודל הערכות-המשתמש של Tsafir). טריק שעוזר כאן: קובץ ה-C מוכלל טקסטואלית לתוך קובץ ה-cdef extern (Cython "m_lublin99.c"), כך ששניהם יחידת קומפילציה אחת והמימוש הסטטי משרת את שניהם.

4.4 הרצת חבילת הבדיקות

בעיה נוספת: pytest 4.6 שמוצמד ב-requirements הוא מ-2019 ולא תואם ל-Python 3.10 (קורס על שינויי AST). שדרוג ל-pytest 7.4 פתר זאת (שינוי בכלי הבדיקה בלבד — לא בקוד הנבדק). התוצאה: **97 מתוך 98 בדיקות עוברות**. הבדיקה הנכשלת (test_packer_scheduler) עוברת כשמריצים אותה לבד — תלות בסדר ההרצה דרך מצב אקראי גלובלי משותף, באג ידוע בבדיקות עצמן ולא בסימולטור.

4.5 הורדת הנתונים האמיתיים

חלק מהבדיקות (והניסויים במאמר) משתמשים ב-trace אמיתי: **LANL-CM5-1994** — יומן של 122,060 משימות אמיתיות ממחשב-העל Connection Machine CM-5 בלוס-אלמוס, בפורמט SWF (Standard Workload Format) מארכיון עומסי העבודה המקבילים של האוניברסיטה העברית. ההורדה האוטומטית בקוד נכשלה (בעיית אישורי SSL בסביבת Python מקומית), אז הקובץ הורד ידנית, נפרס מ-gzip, והונח ב- sched-r1-gym/test/. בבדיקות הפרסור עליו עוברות.

4.6 התאמות Windows לסקריפט האימון

בקובץ deepm-agent.py תוקנו שתי בעיות:

1. **נתיב לינוקסאי:** השורה `TMPDIR = Path(f'/run/user/{os.getuid()}')` משתמשת גם בנתיב שלא קיים ב-Windows וגם בפונקציה `os.getuid` שלא קיימת בו. הוחלף בבדיקת יכולת: אם אין `getuid`, משתמשים בתיקיית ה-temp של מערכת ההפעלה. (התיקייה משמשת להעברת חוויות בין תהליכי העבודה המקבילים — ראו פרק 6.)
2. **תיקיית checkpoints:** הסקריפט שומר את משקולות הרשת ל-`checkpoint/policy-N.pth` אבל לא יוצר את התיקייה — נוספה יצירה אוטומטית.

4.7 baselines, אימון, ותוצאות עד כה

נכתבו שני סקריפטים חדשים בתיקיית הפרוייקט (מחוץ לריפו):

- `eval_baselines.py` — מריץ את היוריסטיקות SJF ואקראי על הסביבה בלי רינדור גרפי, על פני כמה אפיזודות עם seed קבוע, ומדווח תגמול מצטבר ו-slowdown ממוצע.
- `eval_policy.py` — טוען checkpoint של רשת מאומנת ומשווה אותה לאותם baselines בתנאים.

תוצאות ה-baselines (10 אפיזודות, seed 42, עומס deepqm ברירת-מחדל):

סוכן	תגמול מצטבר (ממוצע ± ס"ת)		slowdown ממוצע
SJF	-2317 ± 423	20.0 ± 3.7	
Random	-2749 ± 512	23.3 ± 4.3	

SJF טוב מאקראי, כצפוי — אימות שפיות של הסימולציה.

תוצאות האימון המלא (epochs=200, lr=1e-2, workers × 50 4 מסלולים לעדכון)

סוכן	תגמול מצטבר		slowdown ממוצע
SJF	-2350 ± 408	19.7 ± 3.4	
Random	-2934 ± 580	24.9 ± 4.6	
DeepRM — checkpoint מיטבי (epoch 39)		-2940 ± 660	22.2 ± 5.2

מהלך הלמידה: בתחילת האימון הסוכן גרוע מאקראי (slowdown 27.4 ב-epoch 9), משתפר בהתמדה עד שיא ב-epoch 39 (22.2 — טוב בבירור מאקראי ומתקרב ל-SJF), ואז **מידרדר הדרגתית** חזרה ל-24–26. הדפוס "שיפור ואז קריסה" מוכר היטב ב-REINFORCE וונילה: ללא רגולריזציית אנטרופיה המדיניות הופכת דטרמיניסטית מדי ומאבדת את יכולת החקירה, וקצב למידה קבוע וגבוה (1e-2) מערער את ההתכנסות. אלה בדיוק החולשות שאלגוריתמים מודרניים כמו PPO (יעד 2) נועדו לפתור — כלומר התוצאה הזו מספקת מוטיבציה אמפירית מצוינת להמשך הפרוייקט.

שורה תחתונה ליעד 1: כל הצינור עובד מקצה לקצה — סימולטור, עומסים, סביבה, אימון והערכה — והסוכן אכן לומד (מרמת גרוע-מאקראי לכמעט-SJF). שחזור מלא של עקומות המאמר דורש תקציב אימון גדול יותר ואלגוריתם יציב יותר, שניהם חלק מהתוכנית ממילא.

5. מדריך קוד מלא: sched-rl-gym קובץ-קובץ

5.1 מבט-על: שלוש שכבות

הריפו בנוי בשלוש שכבות נקיות, וזה המפתח להבנתו:

שכבה	תפקיד	קבצים עיקריים
1. סימולטור ליבה	מודל של אשכול: משימות, משאבים, אירועים, תורים. לא יודע כלום על RL.	job.py, resource.py, pool.py, cluster.py, event.py, heap.py, scheduler/*
2. עומסי עבודה	מאיפה מגיעות המשימות: סינתטי פשוט / מודל Lublin / קובץ trace אמיתי	envs/workload.py, workload/*, parallelworkloads
3. סביבת gym	עוטפת את 1+2 בממשק RL סטנדרטי: state, action, reward, step, reset	envs/base.py, envs/deeprm_env.py, envs/compact_env.py, envs/simulator.py

5.2 שכבה 1: המשימה — job.py

מחלקת `Job` מייצגת משימה אחת, עם שדות לפי פורמט SWF: זמן הגשה (`submission_time`), זמן ריצה אמיתי (`execution_time`), מעבדים מבוקשים (`requested_processors`), זמן מבוקש/מוערך (`requested_time`), זיכרון, סטטוס ועוד. בנוסף שדות שהסימולטור ממלא: `start_time`, `finish_time`, `slot_position`. שתי `properties` חשובות:

```
@property
def slowdown(self):
    return (self.finish_time - self.submission_time) / self.execution_time

@property
def bounded_slowdown(self):
    return max(1, (self.finish_time - self.submission_time) / max(10, self.execution_time))
```

אלה בדיוק הנוסחאות מפרק 1. `JobStatus` הוא enum של מצבי החיים: `SUBMITTED` → `SCHEDULED/WAITING` → `RUNNING` → `COMPLETED`.

5.3 שכבה 1: משאבים ואשכול — resource.py, pool.py, cluster.py

האשכול (cluster) מחזיק שני pools: מעבדים וזיכרון. כל pool מנוהל באמצעות IntervalTree (עץ אינטרוולים): הקצאה של משאבים למשימה היא אינטרוול [התחלה, סוף] על ציר יחידות המשאב. כך בדיקת "האם משימה נכנסת עכשיו?" (cluster.fits(job)) ובחירת יחידות פנויות הן פעולות מהירות. מחלקת Resource (ב-resource.py) היא רשימת האינטרוולים שהוקצו למשימה מסוימת.

5.4 שכבה 1: זמן ואירועים — event.py, heap.py

הסימולציה מונעת-אירועים: EventQueue מחזיקה אירועי JobEvent (התחלת משימה / סיום משימה) ממוינים בערימה לפי זמן. כשהזמן מתקדם (job_events.step(1)), כל האירועים שהבשילו "משוחקים" על האשכול: משימות שהסתיימו משחררות משאבים ועוברות לתור completed.

5.5 שכבה 1: המתזמן — scheduler/scheduler.py

מחלקת הבסיס Scheduler מנהלת ארבעה תורים שמתארים את מחזור החיים של משימה:

1. queue_admission — הוגשו, טרם הוחלט עליהן (זה "התור" שהסוכן רואה)

2. queue_waiting — הוחלט לתזמן, טרם התחילו

3. queue_running — רצות עכשיו

4. queue_completed — הסתיימו (מהן מחושבים המדדים הסופיים)

היא גם חושפת properties סטטיסטיות: slowdown (רשימת ה-slowdown של כל המשימות שהסתיימו), load, utilization, makespan. תת-המחלקות (...fifo, sjf, backfilling) מממשות רק את לוגיקת הבחירה.

NullScheduler — הגשר אל ה-RL (scheduler/null_scheduler.py)

זו המחלקה המעניינת ביותר בשכבה 1. היא מתזמן **שלא מחליט כלום בעצמו** — היא מקבלת פקודות מבחוץ (מהסוכן) ומבצעת אותן על הסימולטור:

- step(offset) — "בחרתי את משימה מספר offset בתור". המתזמן בודק אם היא נכנסת לאשכול עכשיו (cluster.fits); אם כן — מקצה משאבים ומוציא אותה מהתור (מחזיר True), אחרת מתעלם (מחזיר False). מספר שלילי = "לא לעשות כלום".
- forward_time() — מקדם את שעון הסימולציה בצעד אחד ומשחק את האירועים שהבשילו.

ההפרדה הזו בין "החלטה" ל"קידום זמן" היא שמאפשרת לסוכן לבצע כמה החלטות באותו צעד זמן: כל עוד הפעולה חוקית והצליחה, הסביבה נותנת לסוכן לבחור שוב בלי להזיז את השעון. רק כשהסוכן בוחר "המתן" או פעולה לא חוקית — הזמן זז.

5.6 שכבה 2: עומסי עבודה — `envs/workload.py`

שלושה מחוללים, נבחרים לפי מילון קונפיגורציה (`workload_config['type']`):

1. `DeepRmWorkloadGenerator` (`type='deeprm'`, **ברירת המחדל**): המודל הסינתטי הפשוט מהמאמר של DeepRM. בכל צעד זמן מגיעה משימה בהסתברות `new_job_rate=0.7`; בהסתברות `small_job_chance=0.8` היא קצרה (1–3 צעדים), אחרת ארוכה (10–15); אחד המשאבים "דומיננטי" (50%–100% מהאשכול) והשני קטן. זה העומס שעליו רץ האימון כרגע.
2. `SyntheticWorkloadGenerator` (`type='lublin'`): עוטף את מודל Lublin99 מהחבילה הקומפלט — עומס ריאליסטי סטטיסטי (זה שהצריך את תיקון ה-`drand48`). אופציונלית מוסיף הערכות-משתמש שגויות לפי מודל Tsafir05 או רעש גאומי — חשוב כי בעולם האמיתי המתזמן לא יודע את זמן הריצה האמיתי, רק את ההערכה.
3. `SwfWorkloadGenerator` (`type='swf'`): מנגן `trace` אמיתי מקובץ SWF (כמו LANL-CM5 שהורדנו).

5.7 שכבה 3: סביבת ה-`envs/base.py` — `envs/deeprm_env.py` ו-`gym`

`BaseRmEnv` היא מחלקת הבסיס שקוראת את כל הקונפיגורציה (`kwargs` ל-`gym.make`): גודל האשכול (10 מעבדים, 10 זיכרון), `time_horizon=20`, `job_slots=5`, `backlog_size=60`, `time_limit=200`, סוג הסימולציה (`simulation_type: time_based/event_based` — זה מתג ה-`γ`, MDP/SMDP!), ועל אילו משימות מחושב התגמול (`reward_jobs`). היא בונה את הרכיבים: מחולל עומס $\leftarrow$ `NullScheduler` $\leftarrow$ `DeepRmSimulator`. פונקציית התגמול:

```
def compute_reward(self, joblist):
    return -np.sum([1 / j.execution_time for j in joblist])
```

בדיוק הנוסחה מפרק 3.1 — כל משימה שנמצאת במערכת תורמת $1/T$ – בכל צעד.

מרחב התצפית של `DeepRmEnv`

שני מצבים, לפי `use_raw_state`:

- **`raw (tuple)`**: (תמונת מעבדים 10×20 , תמונת זיכרון 10×20 , 5 תמונות slots למעבדים, 5 לזיכרון, backlog 3×20 , "זמן מאז משימה אחרונה"). בזה משתמשות ההיוריסטיקות (כמו `sjf-agent`) כי קל לחלץ מזה מידע.
- **`image (ברירת מחדל, Box)`**: כל הרכיבים "נארזים" אופקית למטריצה אחת 124×20 (`pack_observation`) — הקלט לרשת הניורונים.

פונקציית `step` — הלב של הסביבה

זרימת `DeepRmEnv.step(action)`:

1. אם הפעולה היא slot חוקי (0-4) — מתרגמים למיקום בתור (`find_slot_position`); פעולה 5 = המתן.
2. קוראים ל- `simulator.rl_step(action)` — שמעביר ל-`NullScheduler` ואולי מקדם זמן (ראו 5.8). מקבלים בחזרה `intermediate`: רשימת "צילומי מצב" של המשימות בכל צעד-ביניים שעבר.
3. אם המחולל סיים (אין עוד משימות) נזרק `StopIteration` $\leftarrow$ `done=True`. בנוסף יש `time_limit` (ברירת מחדל 200) שעוצר אפיזודות ארוכות.
4. חישוב התגמול: במצב MDP — תגמול בודד על המצב הנוכחי. במצב SMDP — צבירה מנוכה על כל צעדי הביניים:

```
if not done and self.smdp and any(intermediate):
    rewards = [self.compute_reward(js) for js in intermediate]
    reward = (self.gamma ** np.arange(len(intermediate))).dot(rewards)
```

זה בדיוק הסכום $\sum \gamma^k r_k$ מנוסחת ה-SMDP בפרק 2.4.
5. מוחזר: (state חדש, reward, ובסוף אפיזודה — מילון הסטטיסטיקות).

5.8 envs/simulator.py — מתג ה-MDP/SMDP

כאן ההבדל המהותי בין שני המידולים, ושווה להכיר אותו שורה-שורה:

(MDP) `TimeBasedDeepRmSimulator.rl_step`:

```
if self.scheduler.step(action): # action succeeded:
    return [[]]                # no time passes, agent acts again
self.current_time += 1         # otherwise: advance ONE time step
j = self.workload.step()       # maybe a new job arrives
if j: self.scheduler.submit(j)
self.scheduler.forward_time()
return [listjobs()]            # exactly one intermediate state
```

(SMDP) `EventBasedDeepRmSimulator.rl_step`:

```
if self.scheduler.step(action):
    return [[]]
jobs = []
while True:                    # advance time in a LOOP...
    j = self.workload.step()
    if j: self.scheduler.submit(j)
    self.scheduler.forward_time()
    jobs.append(listjobs())     # snapshot per intermediate step
    if self.scheduler.some_job_fits(self.job_slots):
        break                  # ...until a real decision point
return jobs
```

כלומר: בגרסת ה-SMDP, אחרי פעולה שלא הצליחה, הסימולטור **מריץ את הזמן קדימה לבד** עד שמישהי כלשהי מה-slots יכולה להיכנס לאשכול — ורק אז מחזיר שליטה לסוכן, יחד עם צילומי כל צעדי הביניים (שמהם הסביבה מחשבת את התגמול המנוכה המצטבר). הסוכן "מדלג" על כל המצבים חסרי-ההחלטה.

5.9 רכיבים נוספים בקצרה

- `envs/compact_env.py` — ייצוג מצב חלופי קומפקטי (וקטור תכונות לכל משימה במקום תמונה) — בשימוש בחלק מניסויי המאמר.
- `envs/render.py` — ציור מצב הסביבה עם matplotlib (ה-GIF מה-README).
- `schedgym/test_schedgym.py` — 98 בדיקות יחידה על כל השכבות: יצירת משימות, הקצאת משאבים, נכונות כל מתזמן, פרסור SWF ועוד. הרצתן היא הצעד הראשון אחרי כל שינוי.

6. סקריפט האימון REINFORCE — deepm-agent.py לעומק

6.1 הרשת — PGNet

```
fc1: Linear(20*124 → 512) + ReLU
fc2: Linear(512 → 256)      + ReLU
out: Linear(256 → 6)        + Softmax
```

הקלט: תמונת המצב משוטחת לוקטור. הפלט: התפלגות הסתברות על 6 הפעולות (5 slots + המתנה).
`select_action` דוגם פעולה מההתפלגות (`Categorical(probs).sample()`) — דגימה ולא `argmax`, כי האקראיות הזו היא מנגנון החקירה (exploration) של `policy gradient`.

6.2 לולאת האימון — צעד-צעד

1. **איסוף ניסיון במקביל:** N תהליכי עבודה (workers) נפתחים עם `torch.multiprocessing`; כל אחד מריץ `trajectories_per_batch` אפיזודות שלמות עם המדיניות הנוכחית ושומר את המסלולים (state, action, reward לכל צעד) לקובץ pickle בתיקיית temp (זה הקובץ שדרש את תיקון ה-TMPDIR). כל worker מקבל seed שונה (`args.seed + rank`).
2. **חישוב תשואות מנוכות:** במקום לולאה, טריק וקטורי יפה — `make_discount_array` בונה מטריצה משולשת D שבה $D[t,k] = \gamma^{k-t}$ (באמצעות מניפולציית strides של numpy), ואז $G = D \cdot R$ נותן את כל ה- G_t בכפל מטריצות אחד.
3. **advantage ו-baseline:** `compute_baselines` מחשב לכל צעד-זמן t את ממוצע התגמול על פני כל המסלולים באצווה; `advantage` = תשואה מנוכה פחות baseline (מאופס היכן שהמסלול כבר נגמר).
4. **צעד הגרדיאנט:** ההפסד הוא $-\sum \log \pi(a_t|s_t) \cdot A_t - \beta \cdot H(\pi)$, כאשר H הוא רגולריזציית אנטרופיה אופציונלית (`--entropy` , ברירת מחדל 0) שמעודדת חקירה. אופטימיזר: Adam (ברירת מחדל) או RMSprop, learning rate $1e-2$.
5. **ניהול:** callback בשם `ReduceLROnPlateau` מקטין את קצב הלמידה כשההפסד נתקע; כל epoch נשמר checkpoint (`checkpoint/policy-N.pth`) ונכתבים לוגים ל-TensorBoard (היסטוגרמות משקולות, ממוצעי תגמול/advantage/אורכי אפיזודה).

6.3 הסקריפטים שלנו

- `eval_baselines.py` — מימוש headless של לוגיקת `sjf-agent.py` מהריפו: פונקציית `sjf_action` מחלצת מה-state הגולמי את המשימות שנכנסות לאשכול עכשיו ובוחרת את הקצרה; `random_action` מגריל פעולה. רץ על כמה אפיזודות ומסכם.

- `eval_policy.py` — בונה PGNet זהה, טוען משקולות מ-checkpoint, ומריץ את שלושת הסוכנים (מאומן/SJF/אקראי) באותם תנאים בדיוק (אותו seed) להשוואה הוגנת.

7. איך מריצים הכל — פקודות

כל הפקודות מ-PowerShell, מתוך תיקיית הפרוייקט. הסביבה הווירטואלית נמצאת ב-sched-rl--
gym\venv.

בדיקות (אימות הסביבה)

```
cd sched-rl-gym
venv\Scripts\python.exe -m pytest schedgym\test_schedgym.py -q --no-cov
```

בנייה מחדש של מחולל העומסים (אם צריך)

```
venv\Scripts\python.exe -m pip install -e parallelworkloads --no-build-isolation
```

הרצת baselines

```
venv\Scripts\python.exe ..\eval_baselines.py --episodes 10
```

אימון (מהתיקיה sched-rl-gym)

```
venv\Scripts\python.exe deeprm-agent.py --epochs 200 --workers 4 ^
--trajectories-per-batch 50 --max-episode-length 200
```

דגלים שימושיים: --lr (קצב למידה), --entropy (רגולריזציה), --load PATH (המשך מ-checkpoint), --workload FILE.json, (קונפיגורציית עומס אחרת, למשל lublin או swf).

הערכת מדיניות מאומנת

```
venv\Scripts\python.exe ..\eval_policy.py checkpoint\policy-199.pth --episodes 20
```

מעקב אחרי האימון עם TensorBoard

```
venv\Scripts\python.exe -m tensorboard.main --logdir runs
```

מעבר בין MDP ל-SMDP

דרך kwargs של הסביבה — למשל בקובץ workload/קונפיגורציה או בקוד:

```
env = gym.make('DeepRM-v0', simulation_type='event_based', gamma=0.99) # SMDP
env = gym.make('DeepRM-v0', simulation_type='time_based') # MDP
```

8. תוצאות, מסקנות ביניים, וצעדים הבאים

8.1 מצב יעד 1

רכיב	סטטוס
קומפילציה והתקנה של כל הקוד על Windows	הושלם
חבילת בדיקות	97/98 עוברות (הנכשלת — באג סדר-הרצה בבדיקה עצמה)
נתונים: trace אמיתי LANL-CM5 (122,060 משימות)	הורד ועובר פרסור
baselines (SJF, Random)	רצים; SJF 19.7 מול Random 24.9 slowdown ממוצע
אימון REINFORCE (200 epochs)	הושלם; שיא ב-epoch 39: 22.2 slowdown — בין SJF לאקראי
שחזור מלא של גרפי המאמר (MDP מול SMDP)	השלב הבא — דורש אימון ארוך יותר / PPO (יעד 2)

8.2 שאלות פתוחות לפגישה עם גל

- כמה מהקונפיגורציות במאמר לשחזר (עומס trace / lublin / deeprm אמיתי)? אימון על CPU איטי — האם יש גישה ל-GPU?
- אסטרטגיית המיגרציה: לשדרג את הסביבה ל-gymnasium (ה-API החדש) לפני יעד 2, או לעטוף באדפטר?
- ביעד 2 — אילו אלגוריתמים? מועמדים טבעיים: PPO ו-A2C מ-stable-baselines3 (בריפוי יש כבר מחברת PPO אבל עם stable-baselines הישן על TensorFlow 1 — לא שמיש כיום), ואולי DQN עם action masking (הסביבה כבר חושפת `action_masks()`).

8.3 התוכנית ליעדים 2-3

- מיגרציה ל-gymnasium:** שינויי API נקודתיים: reset מחזיר (obs, info); step מחזיר 5 ערכים (terminated/truncated במקום done); רישום סביבות. זה גם "מודרניזציה" שגל ביקש לחשוב עליה.
- אינטגרציה עם stable-baselines3:** אימון PPO על שתי הגרסאות (MDP ו-SMDP) והשוואה ל-REINFORCE ולהיוריסטיקות, על אותם מדדים (slowdown, bounded slowdown, utilization).

3. **יעד 3 — כיוונים אפשריים לאלגוריתם חדש:** שילוב action masking עם SMDP; reward shaping; מבוסס curriculum; bounded slowdown על עומס גובר; או טרנספורמר/attention על תור המשימות במקום ייצוג תמונה. ייבחר עם גל בהמשך.

9. נספח: מילון מונחים

מונח	הסבר
HPC	High-Performance Computing — מחשב-על: אשכולות גדולים למשימות חישוביות כבדות
Job	משימה: בקשת חישוב עם דרישת מעבדים, זיכרון ומשך
Slowdown	(זמן המתנה + ריצה) / זמן ריצה — מדד עיכוב יחסי
Makespan	הזמן עד שהמשימה האחרונה מסתיימת
Utilization	אחוז המעבדים בשימוש בפועל
SWF	Standard Workload Format — פורמט סטנדרטי ליומני עומס של מערכות אמיתיות
Trace	יומן משימות אמיתי שהוקלט ממערכת חיה (כמו LANL-CM5)
Lublin	מודל סטטיסטי (1999) שמייצר עומסים סינתטיים ריאליסטיים
MDP	תהליך החלטה מרקובי — הפורמליזם הבסיסי של RL
SMDP	Semi-MDP — הכללה שבה פעולות נמשכות זמן משתנה; הסוכן מחליט רק בנקודות החלטה
$\pi(a s)$ Policy	המדיניות: התפלגות הפעולות בכל מצב — מה שהרשת לומדת
G Return	סכום תגמולים מנוכה מנקודת זמן ואילך
REINFORCE	אלגוריתם policy gradient בסיסי: חיזוק פעולות לפי התשואה שאחריהן
Baseline / Advantage	ערך ייחוס שמוחסר מהתשואה להקטנת שונות; ההפרש נקרא advantage
Entropy regularization	תוספת להפסד שמעודדת התפלגות פעולות "מפוזרת" — שומרת על חקירה
Epoch / Trajectory	סבב עדכון אחד של הרשת / אפיזודה אחת מלאה של אינטראקציה
Checkpoint	שמירת משקולות הרשת לקובץ באמצע אימון
gym / gymnasium	ממשק סטנדרטי לסביבות RL; gymnasium הוא היורש המודרני של gym
IntervalTree	מבנה נתונים לחיפוש מהיר של חפיפות אינטרוולים — מנהל את הקצאות המשאבים
drand48	מחולל מספרים אקראיים של POSIX (LCG 48-ביט) — מקור האקראיות של מודל Lublin

נוצר ביוני 2026 כחלק מתיעוד פרוייקט הגמר. קבצים נלווים: PROGRESS.md (דוח התקדמות תמציתי), eval_baselines.py, eval_policy.py.